

Data Structures Assignment

PRACTICE QUESTIONS SOLUTIONS

Question 1: What is a Data Structure? Give 3 real-life examples.

Definition: A **Data Structure** is a specialized, systematic format for organizing, managing, processing, and storing data in a computer's memory or storage. It defines the relationship between data elements, the operations that can be performed on them, and ensures that data can be accessed and modified efficiently.

Real-Life Examples:

1. **A Stack of Plates (Stack Data Structure):** In a cafeteria, plates are placed one on top of another. The last plate placed on the stack is the first one to be removed. This mirrors the *Last-In, First-Out (LIFO)* principle of a Stack data structure.
2. **A Queue at a Ticket Counter (Queue Data Structure):** People standing in a line to buy cinema tickets follow a strict order where the person who arrives first is served first. This perfectly represents the *First-In, First-Out (FIFO)* mechanism of a Queue data structure.
3. **An Index Pages of a Textbook (Tree/Index Data Structure):** The table of contents or index at the back of a book organizes chapters and topics hierarchically or alphabetically with page pointers, allowing a reader to jump directly to a topic instead of skimming through every page sequentially. This behaves like an index or Search Tree.

Question 2: Differentiate between Linear and Non-linear Data Structures.

The primary distinction between linear and non-linear data structures lies in how elements are logically arranged and traversed sequence-wise:

Parameter	Linear Data Structure	Non-linear Data Structure
Arrangement	Data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent element.	Data elements are attached hierarchically or interconnected in a non-linear network.
Traversal	Elements can be traversed completely in a single pass (linear run).	Elements require multiple passes or complex algorithms (like DFS/BFS) to traverse fully.
Levels	Single-level data structure.	Multi-level data structure.
Memory Utilization	Can lead to inefficient memory utilization if pre-allocated statically (like arrays).	More flexible and efficient usage of memory for dynamic sets.
Complexity	Simpler to design, implement, and understand.	Comparatively complex to design and implement.
Examples	Arrays, Linked Lists, Stacks, Queues.	Trees (Binary Tree, AVL, Heap) and Graphs.

Question 3: Explain the internal structure of an Array with a memory diagram.

An **Array** is a linear data structure that collects homogeneous data elements (elements of the same data type) in contiguous (adjacent) memory locations. Because memory blocks are contiguous, the exact location of any element can be computed mathematically using its index.

Key properties of an array's internal structure include:

- **Base Address:** The memory location of the very first element (index 0).
- **Index:** A numerical offset value (usually 0 to $n-1$) used to uniquely identify each element position.
- **Element Size:** The uniform number of bytes occupied by the chosen data type (e.g., 4 bytes for a standard integer).

The mathematical formula to find the address of any element at index i is:

$$\text{Address}(A[i]) = \text{Base Address} + (i \times \text{Size of one element})$$

Internal Memory Diagram of an Integer Array (Size = 5, Base Address = 2000):

Index	0	1	2	3	4
Data Value	10	25	42	7	19
Memory Address	2000	2004	2008	2012	2016

Question 4: Write the algorithm and C program for Array insertion.

Insertion into an array involves placing a new data element at a designated target index. Since arrays are stored in contiguous blocks, inserting an element in the middle requires shifting all subsequent elements down by one position to prevent overwriting existing data.

Algorithm for Inserting an Element:

1. Start
2. Check if the array is already full (Overflow Condition). If full, print error and exit.
3. Loop from the last index ($n-1$) down to the target *position*.
4. Shift each element one position to the right: $Array[i+1] = Array[i]$.
5. Insert the new value at the vacant target *position*: $Array[position] = value$.
6. Increment the total element count ($n = n + 1$).
7. Stop

C Program Implementation:

```
#include <stdio.h>

int main() {
    int arr[100] = {10, 25, 42, 7, 19}; // Initialize array with 5 elements
    int size = 5;                       // Current number of elements
    int max_capacity = 100;
    int element, position, i;

    printf("Original Array: ");
    for(i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Values to insert
    element = 99; // Element to be inserted
    position = 2; // Insertion index (0-based)

    // Check for overflow
    if (size >= max_capacity) {
        printf("Error: Array Overflow! Cannot insert element.\n");
        return 1;
    }
}
```

```

}

// Shift elements to the right from the target position
for (i = size - 1; i >= position; i--) {
    arr[i + 1] = arr[i];
}

// Insert the new element at the specified position
arr[position] = element;
size++; // Increase array size by 1

printf("Array After Insertion: ");
for(i = 0; i < size; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

return 0;
}

```

Question 5: Explain Singly Linked List with a node diagram.

A **Singly Linked List** is a dynamic linear data structure composed of a sequence of dynamically allocated elements called **Nodes**. Unlike arrays, nodes are not stored in contiguous memory locations. Instead, they are linked together sequentially via explicit memory pointers.

Each individual **Node** is divided structurally into two parts:

- **Data Field:** Stores the actual payload or value of the element.
- **Next Pointer Field:** Stores the memory address of the succeeding node in the list. The final node's Next pointer points to **NULL**, indicating the list's boundary.

A special pointer called the **Head** keeps track of the memory address of the first node, serving as the entry point for list traversal.

Singly Linked List Node Structure Diagram:

